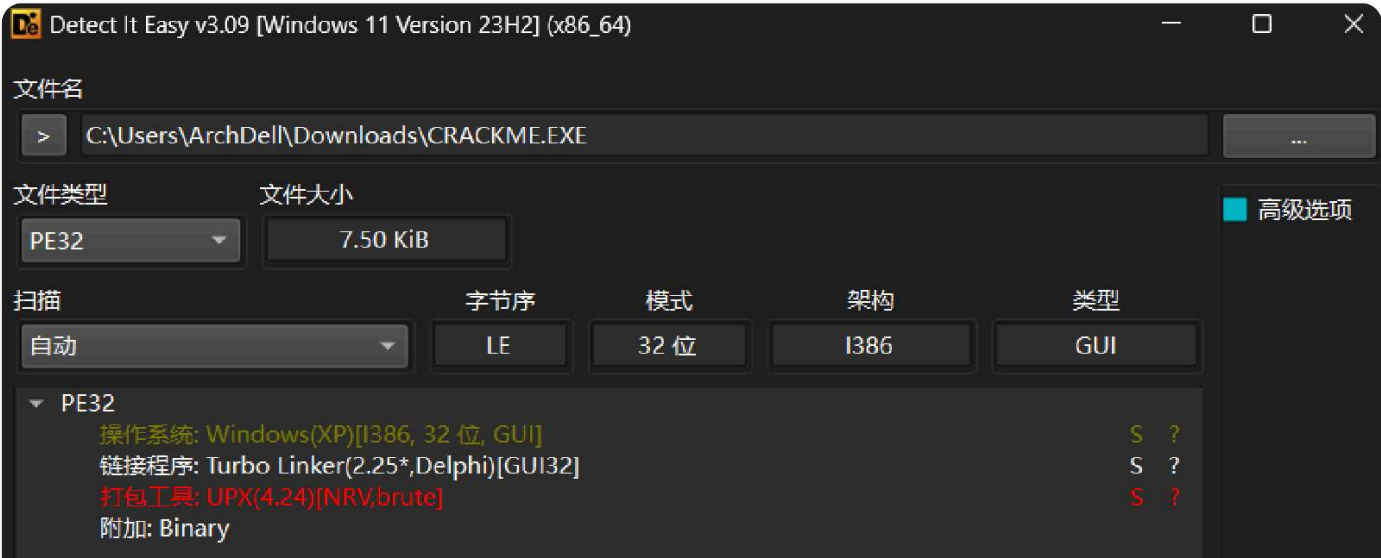


< BACK



Boris's Blog

Act decisively and pursue your desires.



Technology

UPX 脱壳初探 (x64dbg)

UPX 脱壳初探 (x64dbg)

📅 Jun 27, 2024 ⌚ 1705 字

1. Target 程序

CRACKME

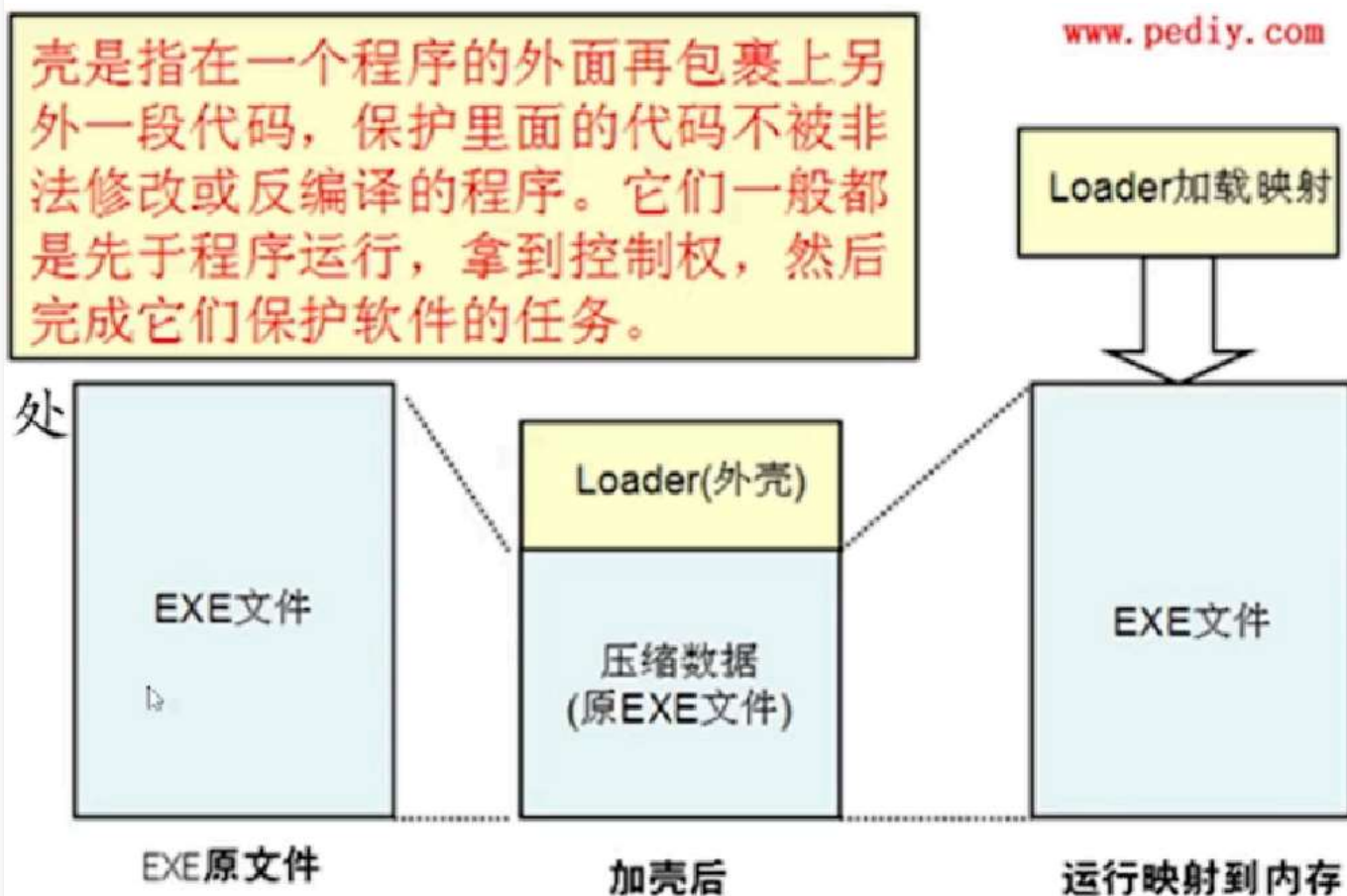
CRACKME UPX

解压密码: 9unk

目标：初步学习脱壳步骤，了解 OEP（程序入口点）和 IAT（Import Address Table）

2. 壳

PE（Portable Executable）也就是 EXE 和 DLL 文件所具有的压缩、加密、保护作用的东西，当然加壳也可以成为我们绕过杀软的一种方式（研究中）



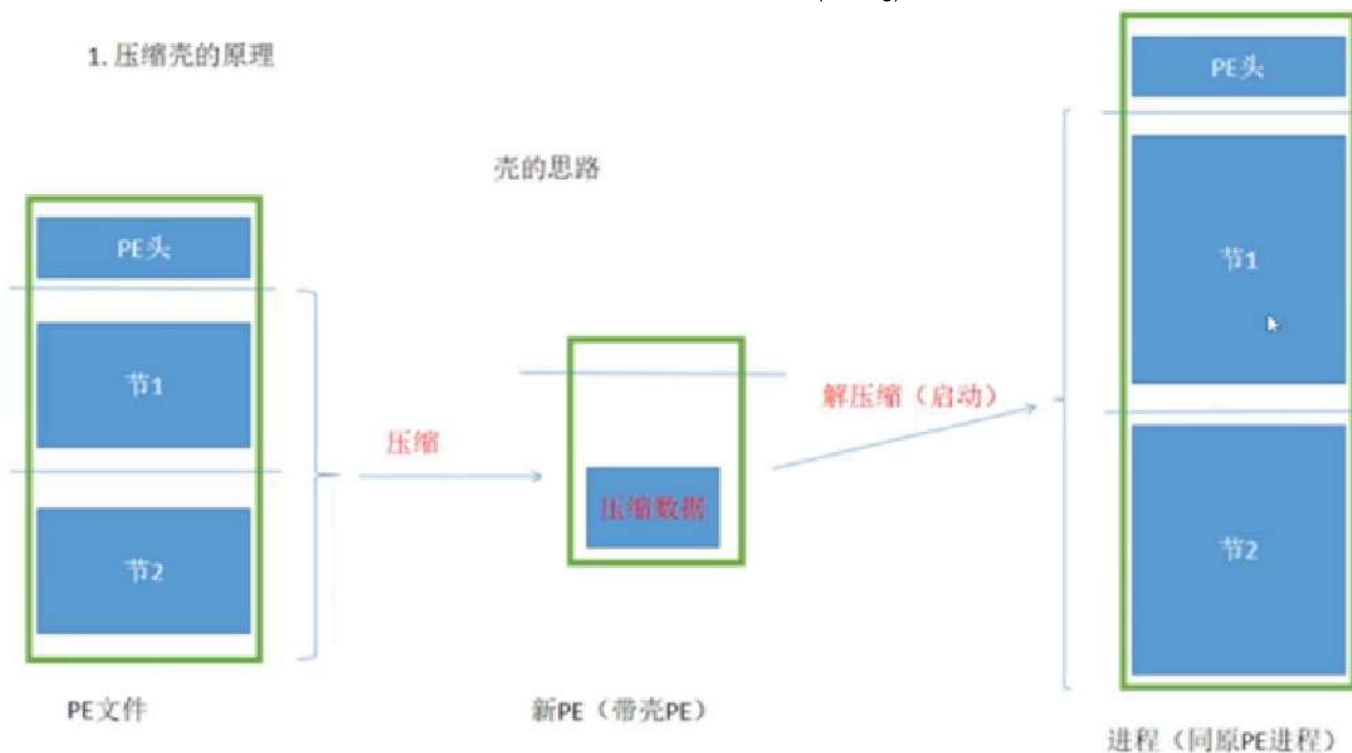
加壳的过程：

1. 将待加壳程序全部进行加密
2. 壳子程序中新增一段解密程序
3. 修改壳子程序的 OEP 到解密程序入口处

3. 壳运行原理（以 UPX 为例）

首先对程序进行解密（解压缩）然后再执行程序

1. 压缩壳的原理



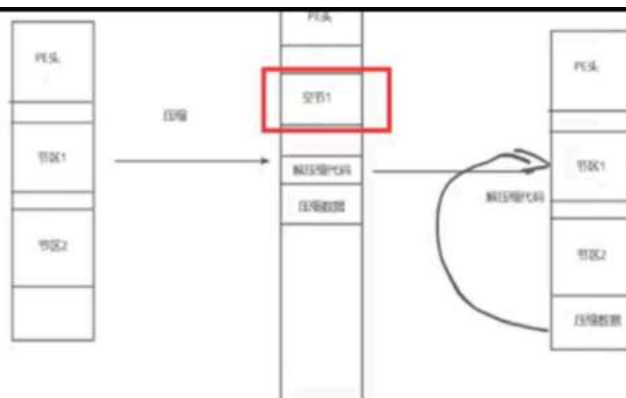
如上图所示，PE文件使用UPX工具进行加壳（压缩），将"节1"和"节2"压缩到"压缩数据"中，并且在"新PE"中增加一段解密（解压）程序，将程序恢复。

4. UPX 加壳原理

UPX 加壳是一种压缩与加密技术，它将可执行文件进行压缩、加密并重新包装，以达到保护程序的目的。加壳后的程序在运行时会自动解压、解密并执行原始程序。这种加壳方式可以有效防止程序被轻易反编译或修改，从而提高软件的安全性。

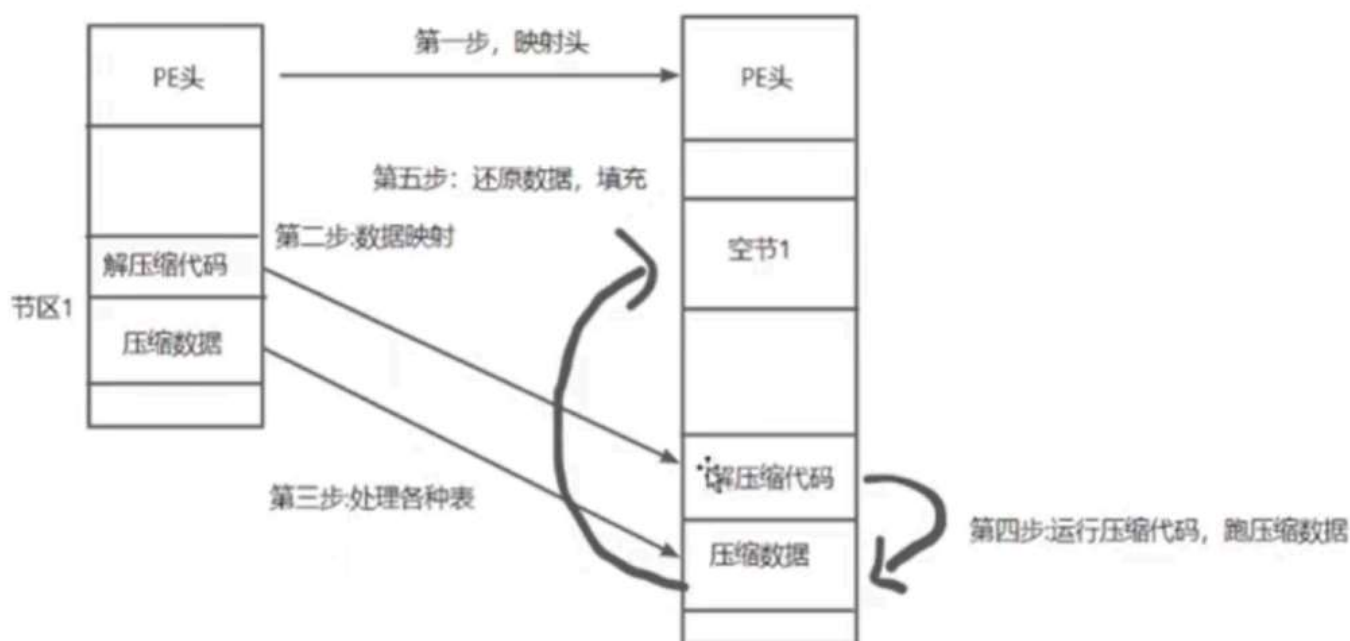
4.1 UPX 加壳过程（UPX 壳，Ultimate Packer for eXecutables，用于可执行文件压缩，减小文件大小，在运行时解压缩）

进程启动后，申请内存，内部包含空节
第一步，先映射头部，当然也要对pe头进行操作
第二步，数据映射，把解压缩代码和压缩数据映射进内存
第三步，把各种表进行处理
第四步，运行压缩代码，跑压缩数据
第五步，还原数据，填充
第六步，执行完解压缩代码后，跑到节区1去执行原来的功能



初始化：进程启动后，申请内存，内部包含空节

1. 先映射头部，当然也要对 pe 头进行操作
2. 数据映射，把解压缩代码和压缩数据映射进内存
3. 把各种表进行处理
4. 运行压缩代码，跑压缩数据
5. 还原数据，填充
6. 行完解压缩代码后，跑到节区 1 去执行原来的功能（会存在大跳）

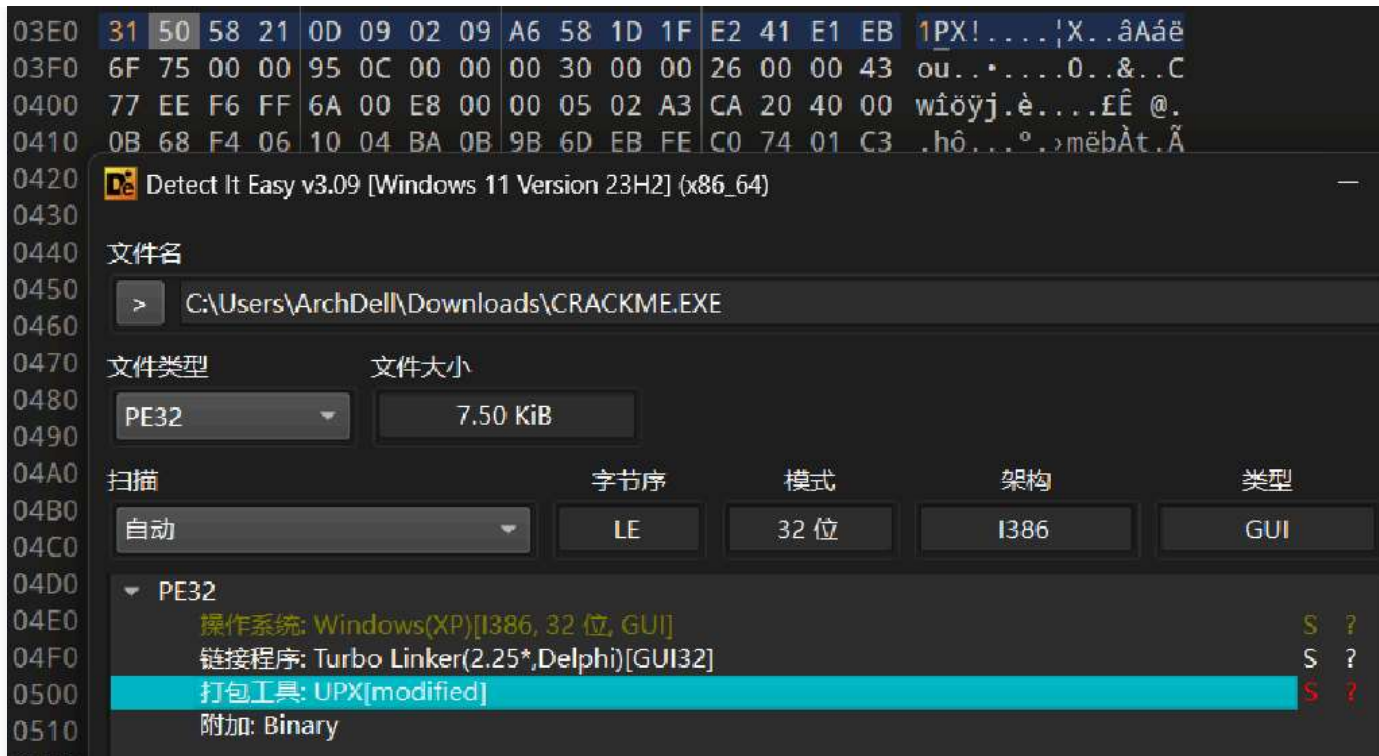


4.2 常见 UPX 魔改情况

区段名被修改 UPX → 1PX 使用 010 Editor 修改回去 如图

03E0	55	50	58	21	0D	09	02	09	A6	58	1D	1F	E2	41	E1	EB	UPX!.....!X..âAâë
03F0	6F	75	00	00	95	0C	00	00	00	30	00	00	26	00	00	43	ou...0..&...C
0400	77	EE	F6	FF	6A	00	E8	00	00	05	02	A3	CA	20	40	00	wîöÿj.è....Ê @.
0410	0B	68	F4	06	10	04	BA	0B	9B	6D	EB	FE	C0	74	01	C3	.hô...°.>mëpÀt.Ă
0420	C7	05	64	0F	03	02	00	09	68	28	11	41	B6	67	9A	40	Ç.d.....h(.A!lgš@

将软件通过 010 Editor 打开，可以发现明确的 UPX 头部，此时使用 Die 可以轻松识别，当我们修改 UPX 头部信息为 1PX 时，此时程序任然可以正常运行，使用 Die 识别到 UPX 但是无法发现具体版本信息，此时使用 UPX -d 进行脱壳是失败的。



```
C:\Users\ArchDell\Downloads>upx -d CRACKME.EXE
Ultimate Packer for eXecutables
Copyright (C) 1996 - 2024
UPX 4.2.4      Markus Oberhumer, Laszlo Molnar & John Reiser    May 9th 2024

File size      Ratio      Format      Name
-----
upx: CRACKME.EXE: CantUnpackException: file is modified/hacked/protected; take care!!!

Unpacked 0 files.
```

5. 脱壳

5.1 直接脱壳

如果能直接识别到 UPX 壳的话，直接使用 UPX -d 进行脱壳如下

```
C:\Users\ArchDell\Downloads>upx -d CRACKME.EXE
Ultimate Packer for eXecutables
Copyright (C) 1996 - 2024
UPX 4.2.4      Markus Oberhumer, Laszlo Molnar & John Reiser    May 9th 2024

File size      Ratio      Format      Name
-----
12288 <-      7680      62.50%      win32/pe      CRACKME.EXE

Unpacked 1 file.
```

5.2 手动脱壳 (x64dbg)

脱壳基本步骤:

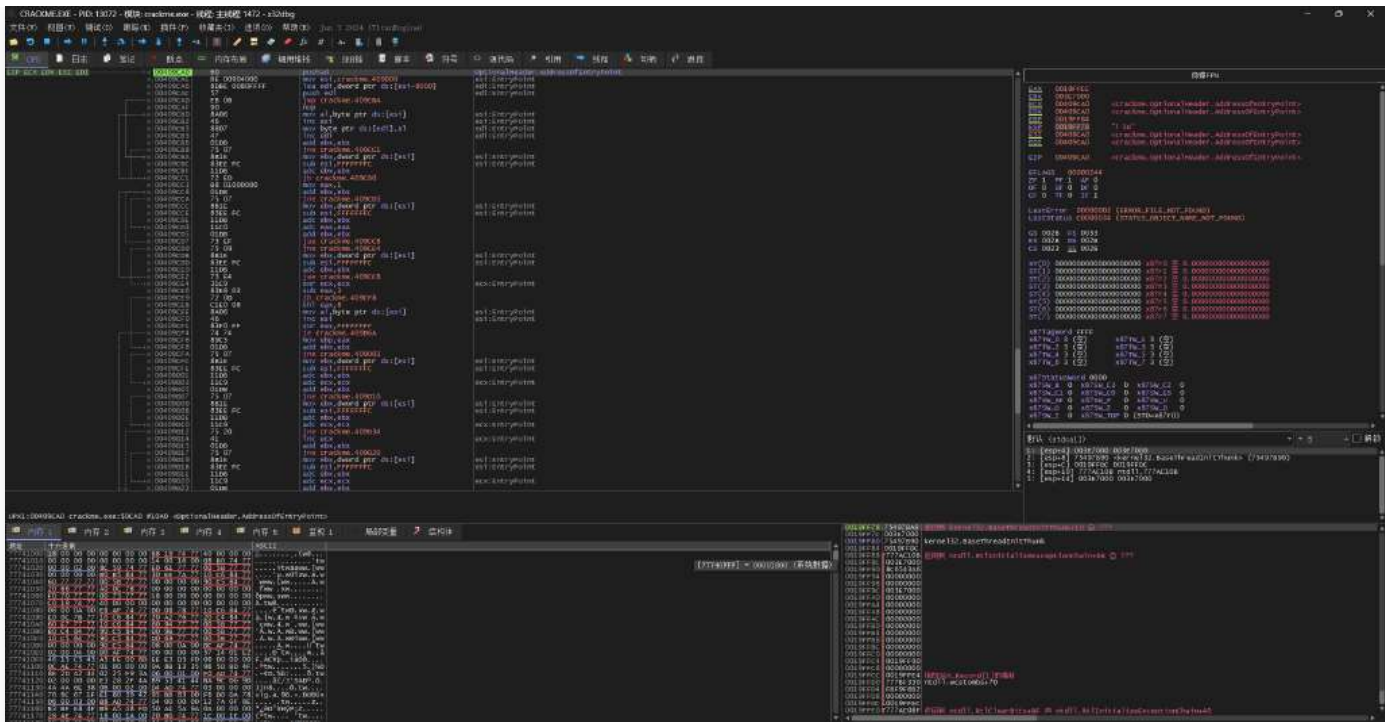
1. 寻找 OEP
2. 转储
3. 修复 IAT (修复导入表)
4. 检查目标程序是否存在 AntiDump 等组织程序被转储的保护措施, 并尝试修复这些问题。

寻找 OEP:

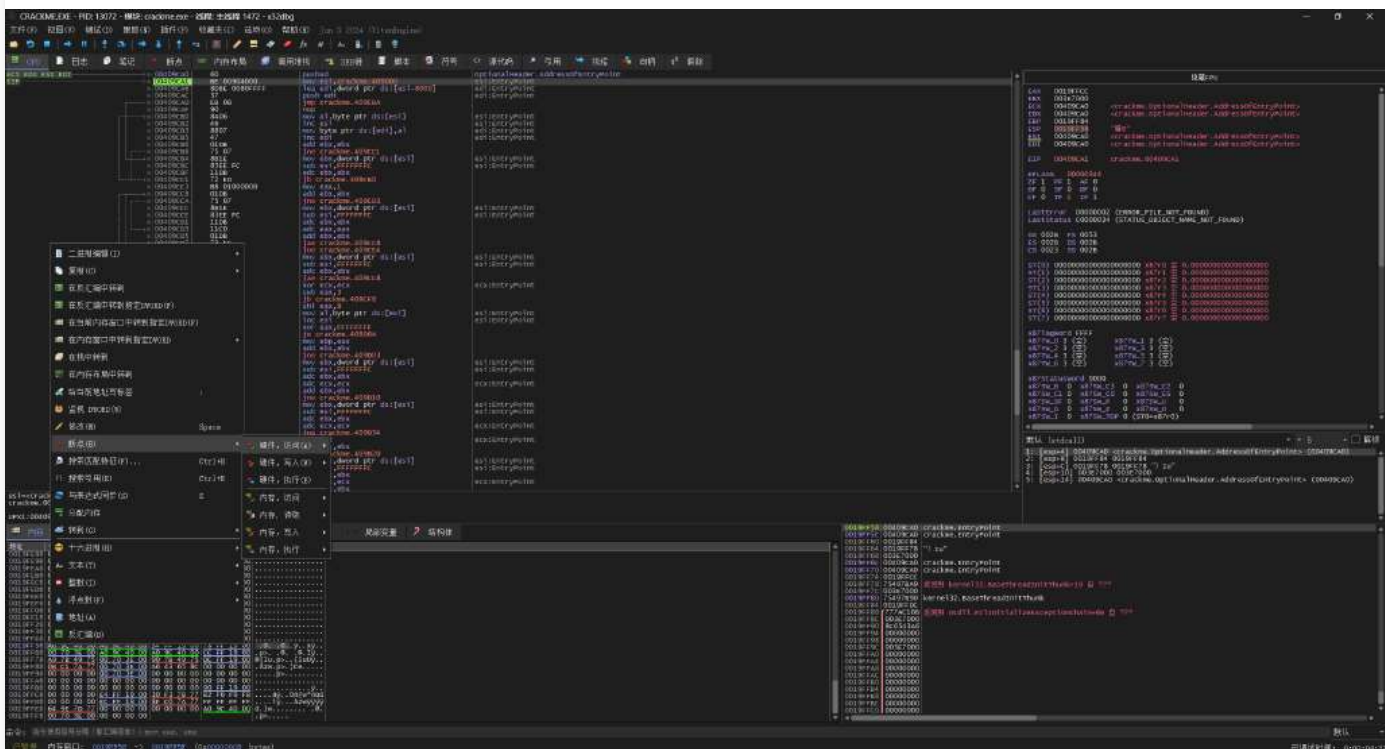
1. 单步跟踪法: 主要使用 “F8” 和 “F4” 这两个快捷键, 一步一步分析每一条汇编背后所代表的意思, 将壳代码读懂, 从而找到原始 OEP 然后脱壳。
2. ESP 定律法: 一般加壳程序在运行时, 会先执行壳代码, 然后在内存中恢复还原原程序, 再跳转到原始 OEP, 执行原程序的代码。这些壳代码首先会使用 PUSHAD 指令保存寄存器环境, 在解密各个区段完毕, 跳往 OEP 之前会使用 POPAD 指令恢复寄存器环境。
3. 内存镜像法: 在加壳程序执行时, 会先将源程序的 “CODE” 和 “DATA” 区段解压 \ 解密并载入内存, 然后再载入 “rsrc” 资源到内存中, 最后跳到 OEP 执行解密后的程序。内存镜像法就是在 rsrc 先设置一个内存执行断点, 当程序停下来的时候说明程序已经解压 \ 解密完成。此时再到 “DATA” 区段设置内存执行断点, 程序下一次会停在 OEP 入口点。
4. 一步到达 OEP (😄.jpg): 使用快捷键 “Ctrl+B” 搜索十六进制字符串 “E9 ?? ?? ?? ?? 00 00 00 00 00 00 00 00 00 00 00 00 00 00”, 即可找到跳转 OEP 的位置; 使用快捷键 “Ctrl+F” 搜索 popad。找的 popad 需要满足, 在程序返回时, 壳程序希望恢复现场环境的地方。也就是靠近 jmp 和 return 的地方。

5.2.1 ESP 定律

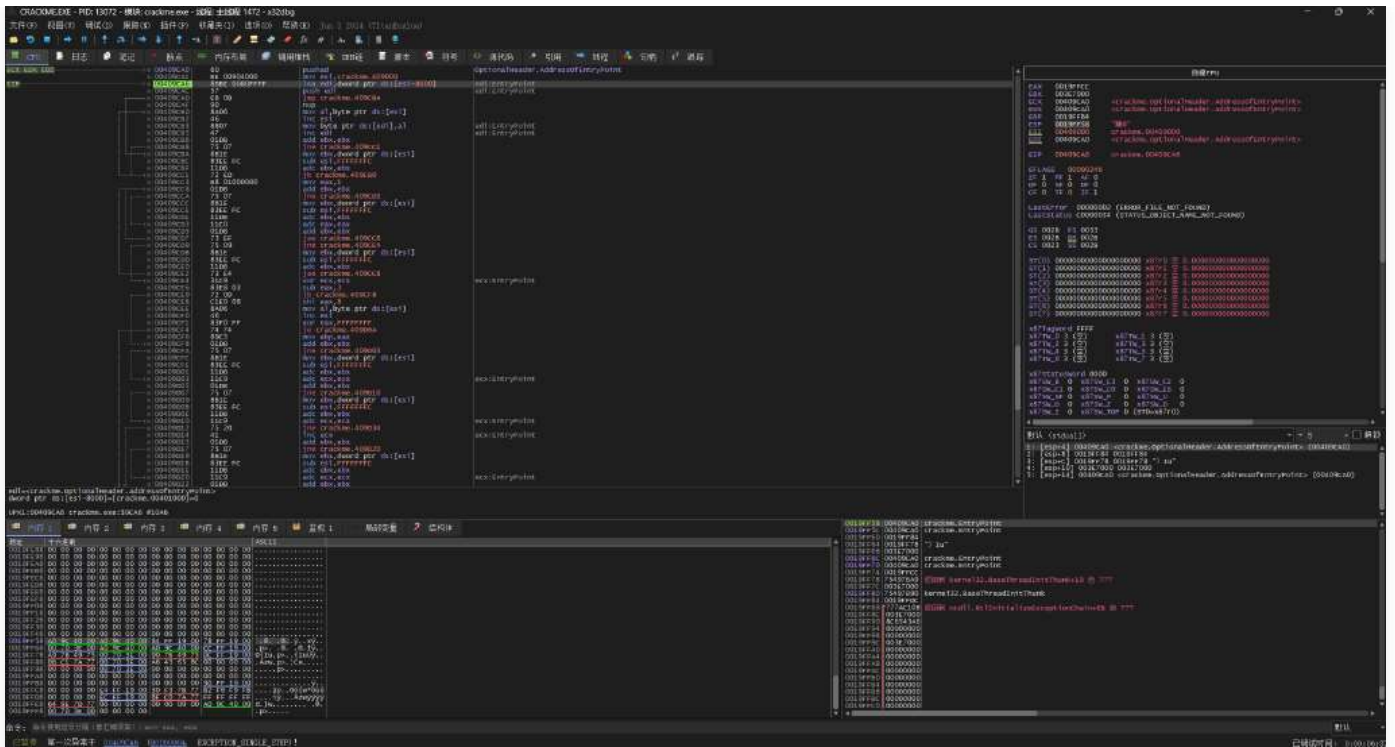
1. F9, 运行到 pushad 指令, F7 执行到下一行汇编指令, 此时寄存器的值存储到栈顶, 也就是 esp 的位置。



2. 右击 esp 寄存器，选择在内存中跳转。此时才内存窗口可以看到当前存储的寄存器值，并如图设置硬件访问断点。

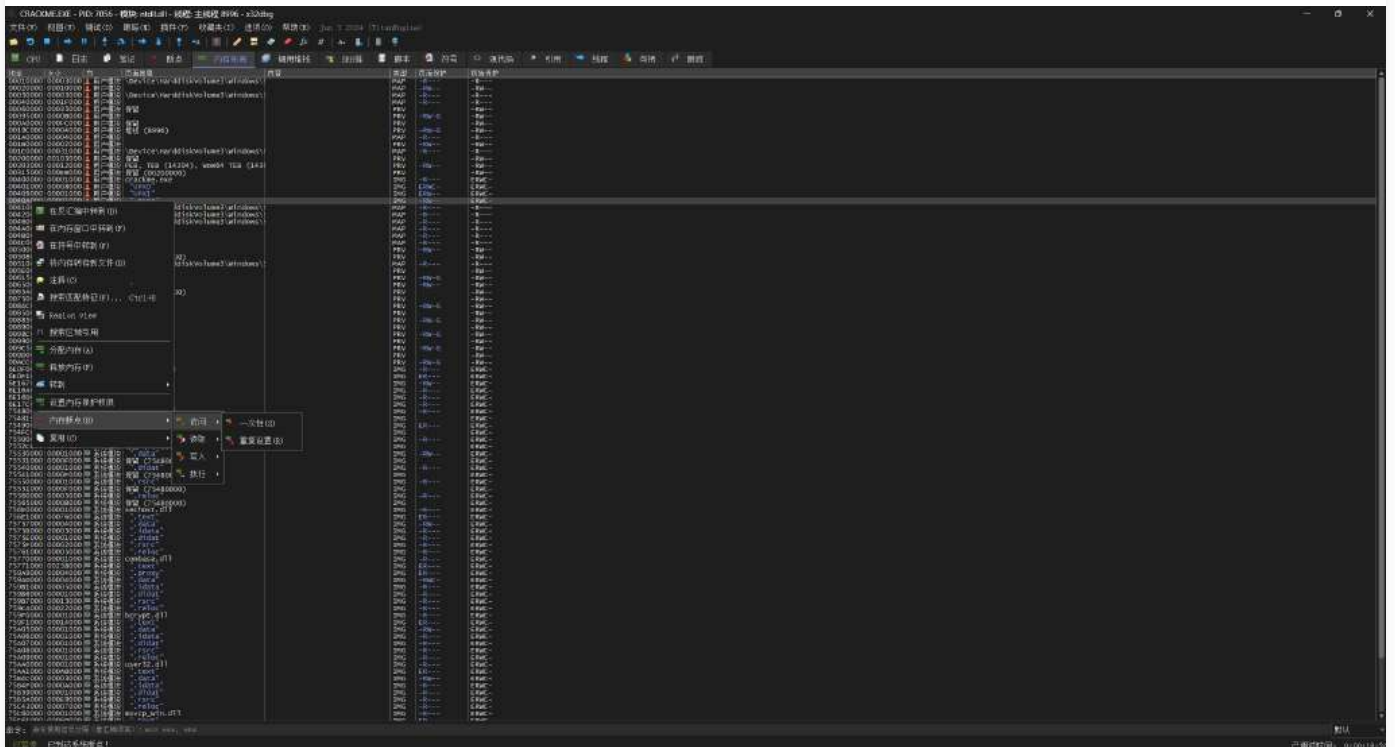


3. “F9” 运行程序后，程序断在如下位置。下面有个 jmp 指令，跳转到 OEP 处。OEP 地址为：0x00409CAD

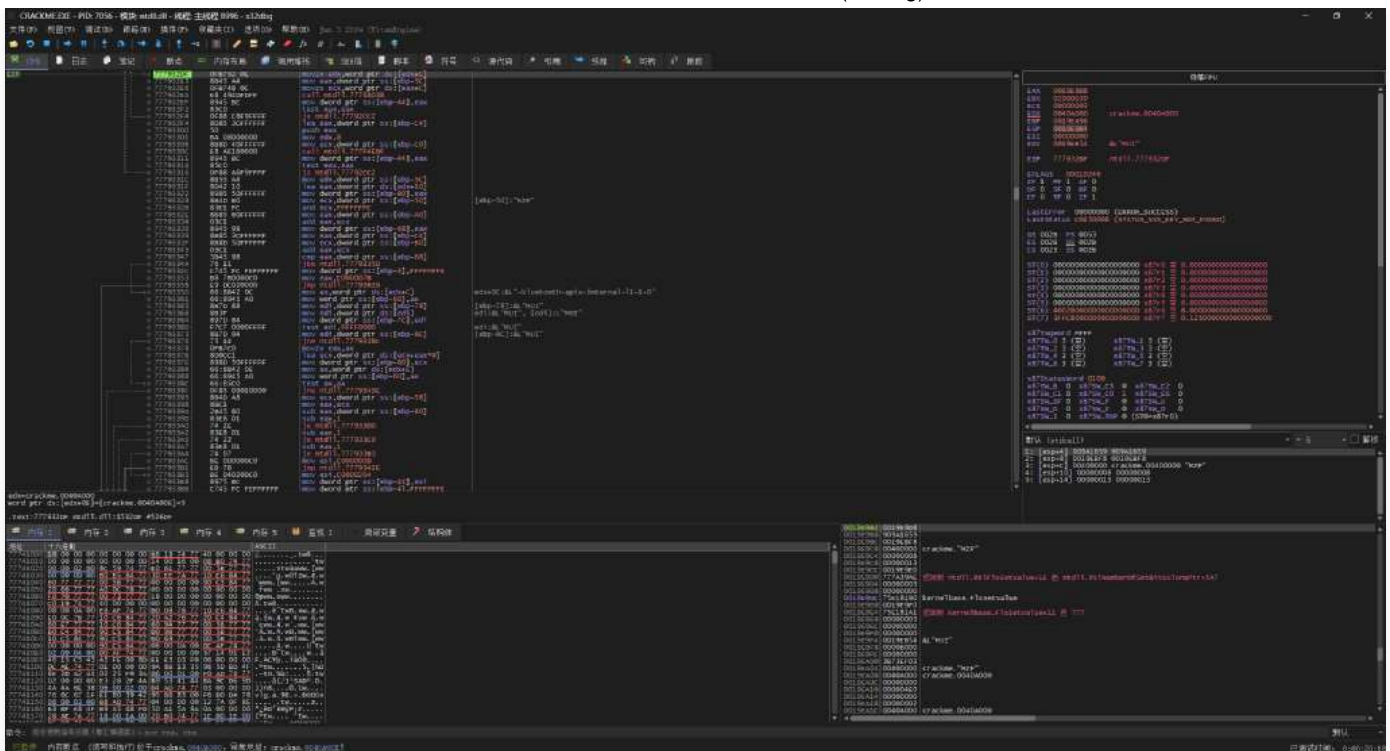


5.2.2 内存镜像法

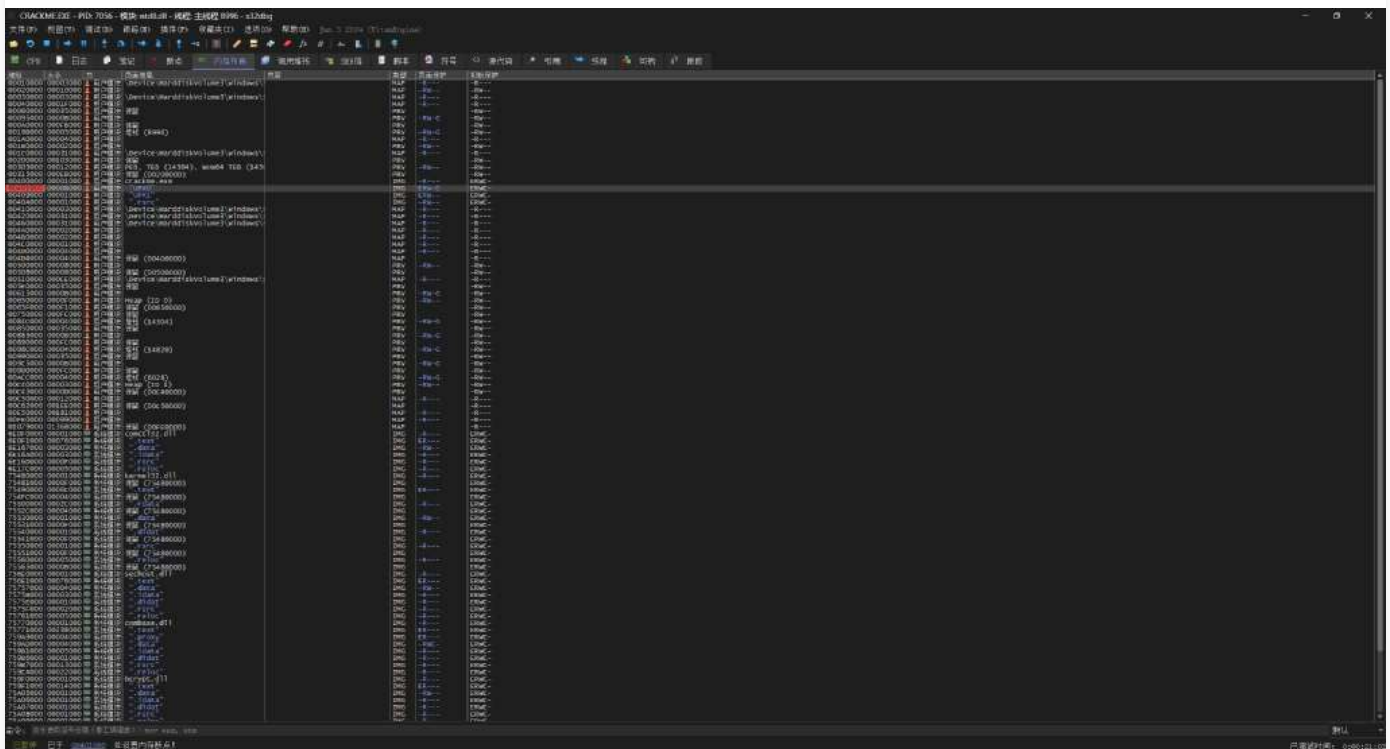
1. 载入 UPX 程序，使用快捷键 “Alt+M” 进入到内存视图，对 “.rsrc” 区段设置内存访问断点



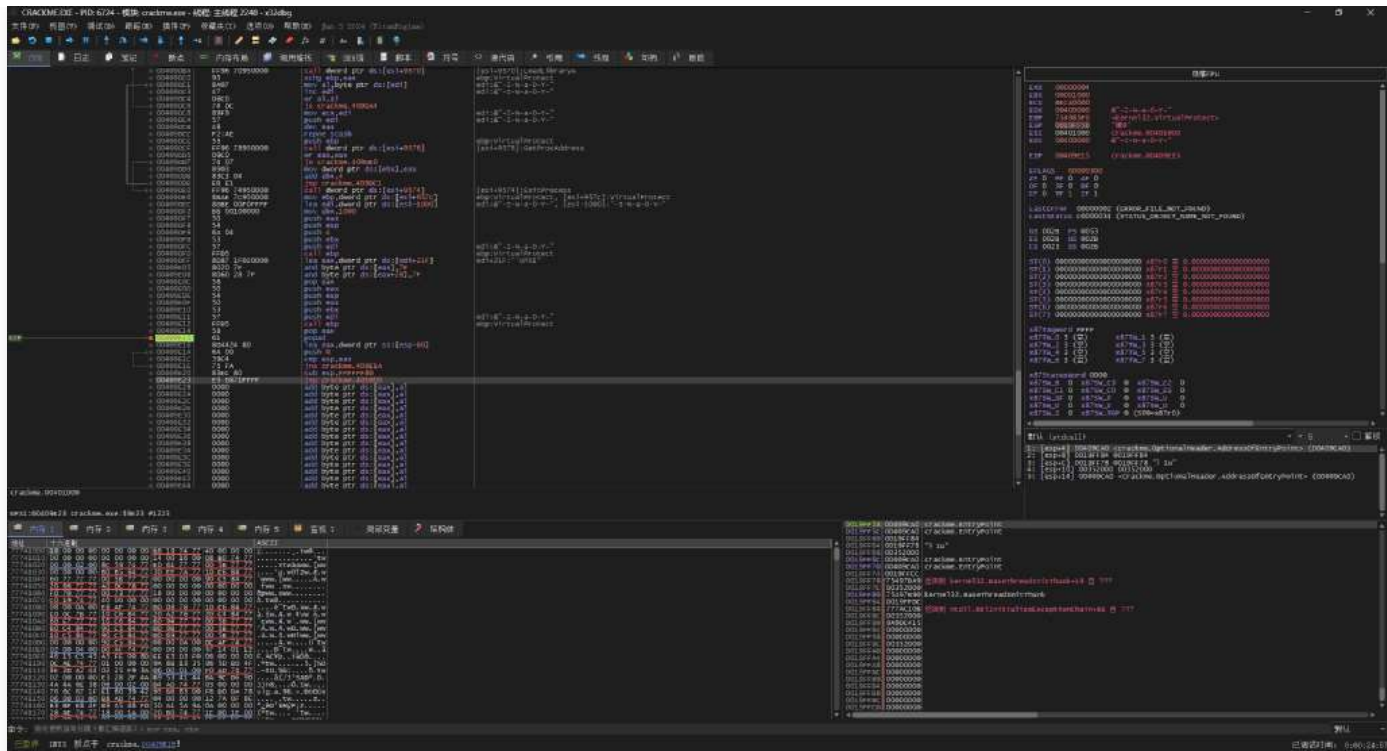
2. “F9” 运行 upx 程序到 “.rsrc” 区段，此时前面两个区段已经解密好了。



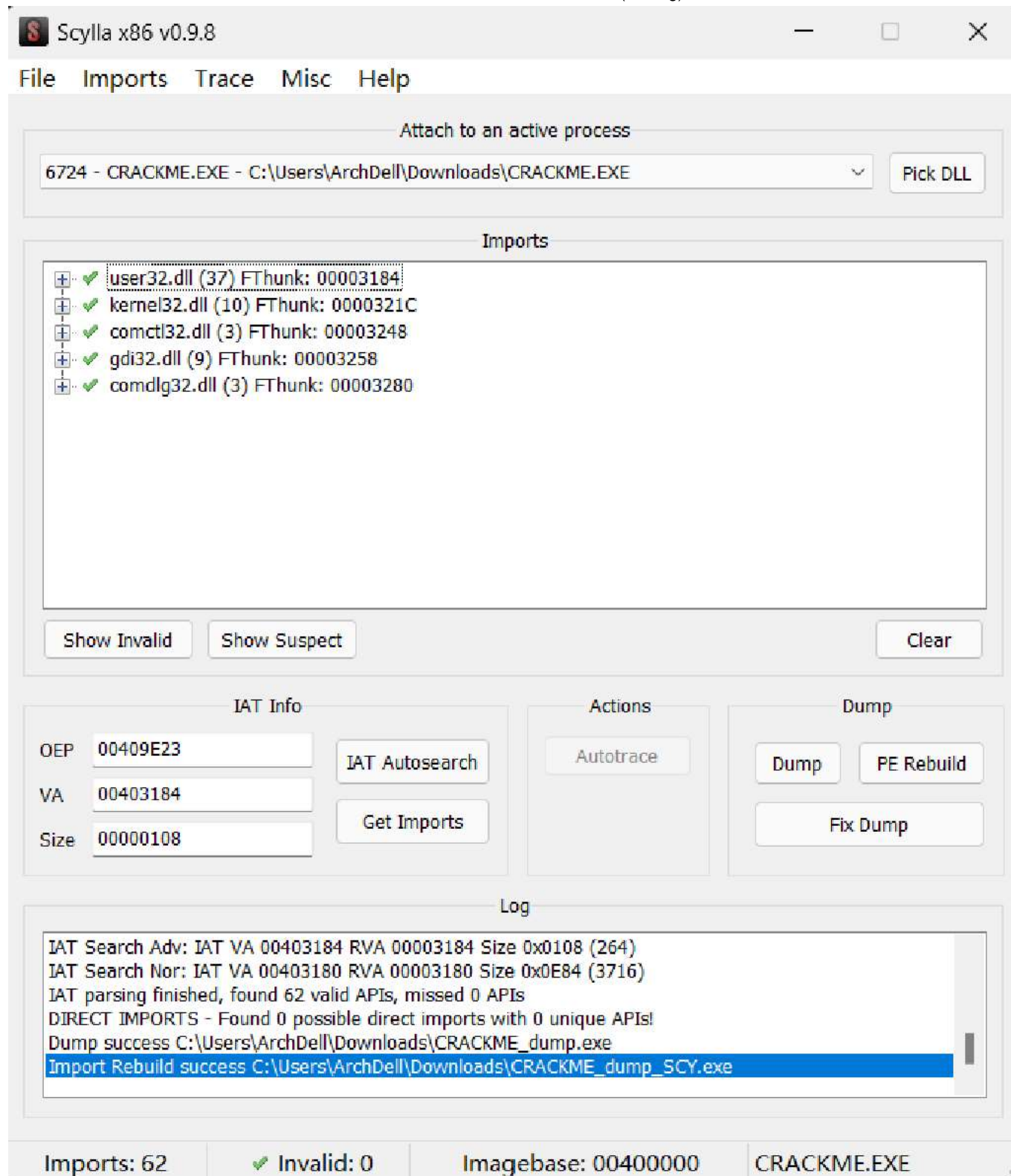
3. 再次到内存视图，使用 “F2” 对 “CODE” 区段设置内存执行断点。



4. 继续 “F9” 执行代码，此时可以看到程序停在了 OEP 入口处 (0x00409E23)

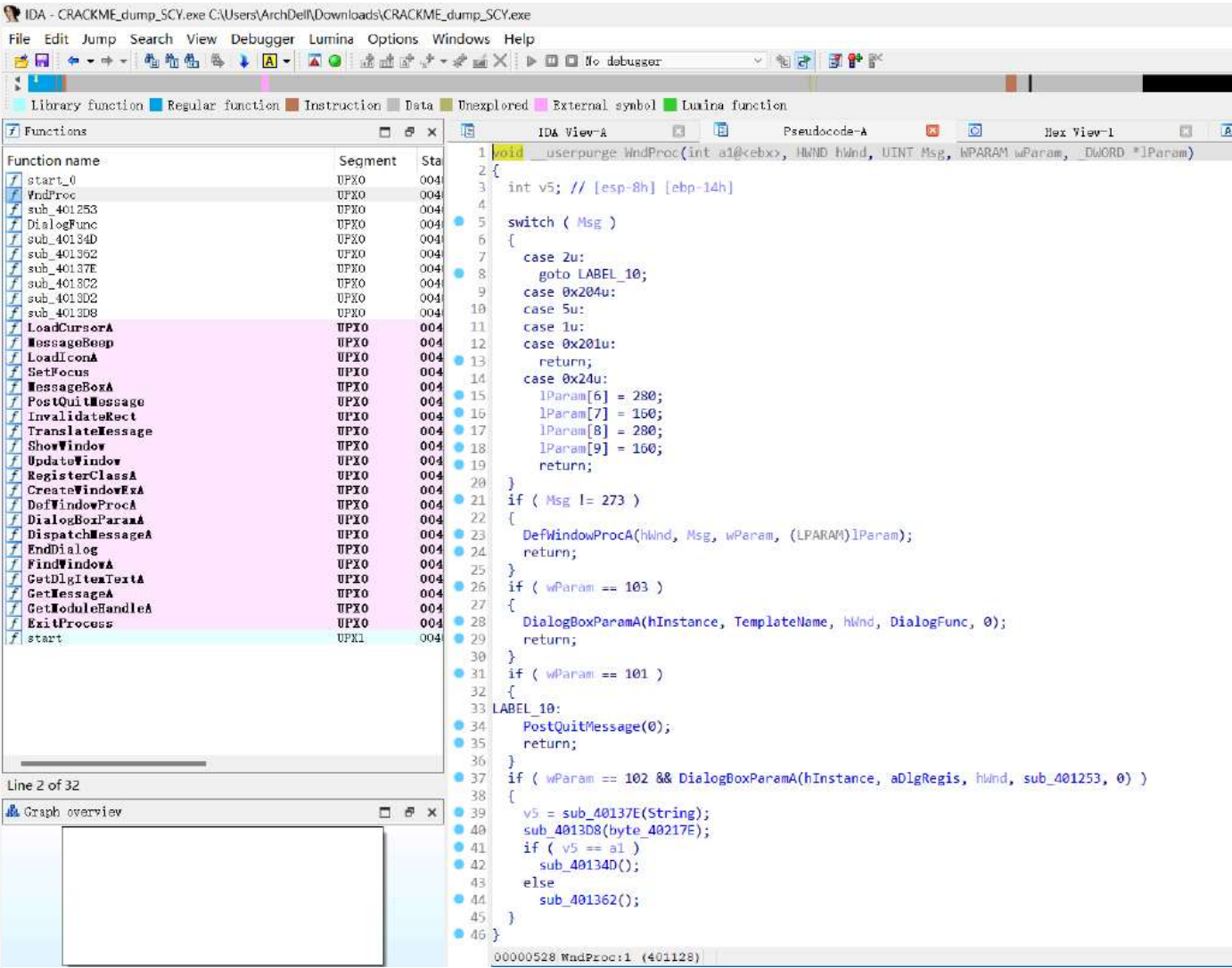


6. 修复 IAT 表



设置 OEP 地址 -> IAT Autosearch -> Get Imports -> Dump -> Fix Dump

😊 CRACKME_dump_SCY.exe	2024/6/28 14:27	应用程序	38 KB
😊 CRACKME_dump.exe	2024/6/28 14:26	应用程序	37 KB
😊 CRACKME.EXE	2024/6/28 13:44	应用程序	8 KB



Reverse UPX

© LICENSED UNDER CC BY-NC-SA 4.0

DLL 注入

Go 语言 ShellCode 免杀火绒

PwnCollege 学习

© 2024 Boris's Blog · 796Days
共书写了 7.9k 字 · 共 5 篇文章

Built with Hugo

Theme **Stack** designed by Jimmy

© Licensed Under CC BY-NC-SA 4.0